2026-2027 学年第一学期

《高级语言程序设计(C++语言)》

实验报告

实验名称：	数据的共享与保护（实验三）		
实验日期：	2026.11.13	实验地点：	于刚宋晓楼 A237
学　　号：	2025302183225	姓　　名：	徐维鑫
学　　院：	人工智能学院	专　　业：	人工智能专业
指导老师：	丁建立		

一、实验目的

1. 观察程序运行中变量的作用域、生存期和可见性；

2. 学习类的静态数据成员和静态成员函数的使用；

3. 学习多文件结构在 C++ 程序中的使用；

4. 理解友元函数、友元成员函数和友元类的定义与使用；

5. 掌握类声明（.h）、类实现（.cpp）与测试主函数（main.cpp）分离的项目组织方式；

6. 养成先分析、再编码、编译开警告、运行后验证的习惯。

二、实验环境

操作系统：Windows 11（版本 10.0.26200）。

编译器：MinGW-w64 GCC 8.1.0(g++)，编译选项 -std=c++11 -Wall -Wextra -fexec-charset=GBK。

编辑器：Visual Studio Code + C/C++ 扩展（ms-vscode.cpptools）。

项目组织：每个题目单独一个目录；类声明放在 .h 头文件，类实现放在 .cpp 源文件，测试主函数放在 main.cpp；编译时把同一目录下的所有 .cpp 一起交给 g++。

三、实验过程及结果

本次实验按照“课堂练习（静态数据成员、静态成员函数）→ 编程练习（作用域与生存期、静态成员、友元与多文件结构）”展开，共完成 6 个程序，并书面回答思考题 5-16。每个题目单独一个目录，类声明、类实现和测试主函数分文件存放，全部源码位于仓库 work3/src/ 目录。下面按练习顺序给出各题目的代码、运行结果与分析。

课堂练习一 静态数据成员

1. 静态数据成员的定义与使用

实验内容：按课件程序创建 Ctype 类，a 为普通数据成员、s 为私有静态数据成员；在类外定义并初始化 s，创建 c1、c2、c3 后依次调用 Print，观察 a 和 s 的变化。

代码：

【Ctype.h】

```
#ifndef CTYPE_H
#define CTYPE_H

class Ctype {
private:
    int a;                // 普通数据成员：每个对象各有一份
    static int s;         // 静态数据成员：所有 Ctype 对象共享一份

public:
    void Print();
    Ctype(int x = 0);     // 默认参数只在声明中写一次
};

#endif  // CTYPE_H
```

【Ctype.cpp】

```
#include <iostream>
#include "Ctype.h"
using namespace std;

// 静态数据成员必须在类外定义并初始化，定义时前面不能加 static。
// 这一行同时为 s 分配存储空间，并把它初始化为 0。
int Ctype::s = 0;

// 构造函数定义中不能再写默认参数 =0，否则会报“默认参数重定义”。
Ctype::Ctype(int x) {
    a = x;
}

void Ctype::Print() {
    cout << "a=" << ++a << endl;   // a 是普通成员，每个对象各有一份
    cout << "s=" << ++s << endl;   // s 是静态成员，三个对象共用
}
```

【main.cpp】

```
#include "Ctype.h"

int main() {
    Ctype c1, c2, c3;

    // 三个对象的 a 互不影响，但 s 是同一个静态数据成员。
    c1.Print();
    c2.Print();
    c3.Print();

    return 0;
}
```

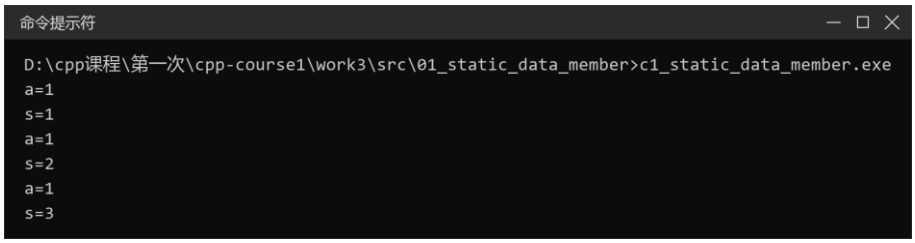


图 1-1 静态数据成员的定义与使用运行结果

结果分析：运行结果为 a=1、s=1，a=1、s=2，a=1、s=3。每个对象的 a 各自从 0 开始，Print 中 ++a 后都为 1；s 是静态数据成员，三个对象共享同一份存储，++s 依次得到 1、2、3。静态数据成员不属于某个对象，必须在类外定义和初始化，定义时前面不能加 static。另外，课件中构造函数的定义如果重复写 int x=0，会报“默认参数重定义”，实现文件中应去掉默认值，只在声明中保留。

课堂练习二　静态数据成员与普通数据成员的区别

1. 学生计数

实验内容：定义 Student 类，用静态数据成员 count 统计学生总数，用普通数据成员 StudentNo 保存每个学生的学号；每创建一个对象，count 加 1，学号取 count 的当前值。

代码：

【Student.h】

```
#ifndef STUDENT_H
#define STUDENT_H

class Student {
private:
    static int count;      // 静态数据成员：统计学生的总数
    int StudentNo;         // 普通数据成员：每个学生自己的学号

public:
    Student();
    void Print();
};

#endif  // STUDENT_H
```

【Student.cpp】

```
#include <iostream>
#include "Student.h"
using namespace std;

// 静态数据成员在类外定义并初始化，前面不能加 static。
int Student::count = 0;

Student::Student() {
    ++count;               // 每构造一个对象，共享的 count 加 1
    StudentNo = count;     // 学号取当前总数：1、2、3、4
}

void Student::Print() {
    cout << "Student" << StudentNo << " ";
    cout << "count=" << count << endl;
}
```

【main.cpp】

```
#include <iostream>
#include "Student.h"
using namespace std;

int main() {
    Student Student1;
    Student1.Print();
    cout << "----------" << endl;

    Student Student2;
    Student1.Print();
    Student2.Print();
    cout << "----------" << endl;

    Student Student3;
    Student1.Print();
    Student2.Print();
    Student3.Print();
    cout << "----------" << endl;

    Student Student4;
    Student1.Print();
    Student2.Print();
    Student3.Print();
    Student4.Print();

    return 0;
}
```



图 2-1 学生计数运行结果

结果分析:StudentNo 是普通数据成员,每个对象各有一份,构造时取 count 的当前值，所以依次为 1、2、3、4；count 是静态数据成员，所有对象共享，随着对象创建变为 1、2、3、4。输出中 Student1 的学号始终是 1，而 count 不断增大，说明“静态成员属于类、普通成员属于对象”。课件原程序中的 count<<、Student1.print() 等笔误，以及重复定义 Student3、未定义的 Student4，已按上下文修正为 cout、Student1.Print()，并补上 Student4 的创建。

课堂练习三　静态成员函数

1. 找出程序中的错误并改正

实验内容：课件程序用 static void Print() 演示静态成员函数，要求找出错误并上机验证。原程序在静态函数中访问普通数据成员 a，并在创建任何对象之前用类名调用 Print。

代码：

【课件原程序（节选，错误处已标注）】

```
class Ctype {
private:
    int a;
    static int s;
public:
    static void Print();      // 静态成员函数
    Ctype();
};

void Ctype::Print() {
    cout << "a=" << ++a << endl;   // 错误：静态函数没有 this，不能访问 a
    cout << "s=" << ++s << endl;   // 正确
}

Ctype::Ctype() {
    a = 0;
    s++;
    cout << "a=" << ++a << endl;
}

int Ctype::s = 0;

int main() {
    Ctype::Print();          // 创建对象前用类名调用
    Ctype c1, c2;
    c1.Print();
    c2.Print();
    Ctype c3;
    c3.Print();
    return 0;
}
```

【Ctype.h（改正后）】

```
#ifndef CTYPE_H
#define CTYPE_H

class Ctype {
private:
    int a;
    static int s;          // 私有静态数据成员

public:
    static void Print();   // 静态成员函数：没有 this 指针
    void PrintA();         // 普通成员函数：可以通过对象访问 a
    Ctype();
};

#endif  // CTYPE_H
```

【Ctype.cpp（改正后）】

```
#include <iostream>
#include "Ctype.h"
using namespace std;

int Ctype::s = 0;

Ctype::Ctype() {
    a = 0;
    s++;
    cout << "a=" << ++a << endl;
}

void Ctype::Print() {
    // 错误原因：静态成员函数没有 this 指针，不能直接访问非静态成员 a。
    // 原程序中的 cout<<"a="<<++a<<endl; 会编译报错。
    // 这里只访问静态成员 s，因此这个函数合法。
    cout << "s=" << ++s << endl;
}

void Ctype::PrintA() {
    // 普通成员函数有 this 指针，可以访问非静态成员 a。
    cout << "a=" << ++a << endl;
}
```

【main.cpp（改正后）】

```
#include <iostream>
#include "Ctype.h"
using namespace std;

int main() {
    // 还没有定义任何对象，可以直接通过类名调用静态成员函数。
    Ctype::Print();

    Ctype c1, c2;
    c1.Print();        // 也可以用对象调用静态成员函数
    c2.Print();

    c1.PrintA();       // 普通成员函数必须通过对象调用
    c2.PrintA();

    Ctype c3;
    c3.Print();
    c3.PrintA();

    return 0;
}
```

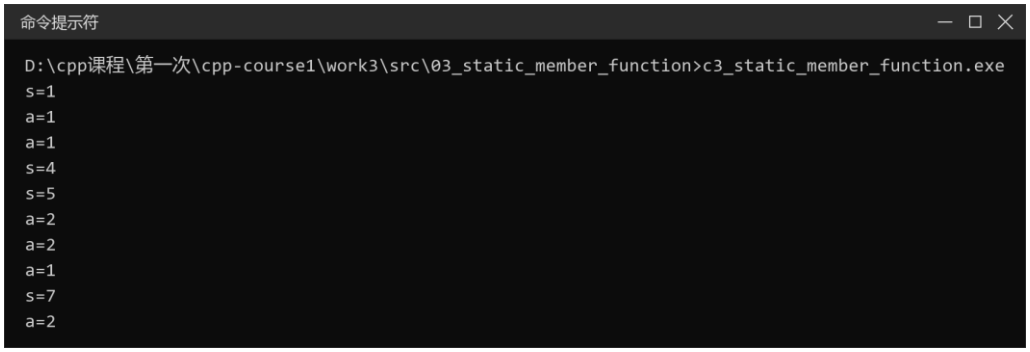


图 3-1 找出程序中的错误并改正运行结果

结果分析：原程序有两处问题。第一，static void Print() 中写了 ++a：静态成员函数没有 this 指针，不能直接访问非静态数据成员 a，GCC 报 invalid use of member 'Ctype::a' in static member function。第二，Ctype::Print() 在任何对象创建之前调用；如果 Print 仍访问 a，此时没有对象可以提供 a。把 Print 改为只访问静态成员 s 后，这一调用就合法了，说明静态成员函数可以在没有对象时通过类名调用。改正方案是:Print 保留 static，只输出 ++s；另加普通成员函数 PrintA()，通过对象输出 ++a。运行结果依次为 s=1（类名调用）、a=1、a=1（c1、c2 构造）、s=4、s=5（c1、c2 调用 Print）、a=2、a=2（调用 PrintA）、a=1（c3 构造）、s=7（c3 调用 Print）、a=2（c3 调用 PrintA）。此外，s 既在构造函数中自增又在 Print 中自增，如果 s 的本意是对象个数，Print 中不应再自增，这是语义问题而不是编译错误；若确实要在静态函数中访问某个对象的 a，可以把对象作为参数传入，例如 static void Print(Ctype &c)。

编程练习 5-3 变量的作用域、生存期与可见性

1. 全局变量与局部变量的作用域

实验内容:程序定义全局变量 x=5、y=7;myFunction 中定义局部变量 y=10,并在函数内、外分别输出 x、y，观察同名变量的遮蔽和函数返回后的变化。

代码:

【main.cpp】

```
#include <iostream>
using namespace std;

int x = 5, y = 7;          // 全局变量：作用域从定义处到文件末尾，生存期贯穿整个程序

void myFunction() {
    int y = 10;            // 局部变量，遮蔽（隐藏）同名的全局变量 y
    cout << "x from myFunction " << x << "\n";
    cout << "y from myFunction " << y << "\n\n";
}                          // 局部 y 离开作用域后被销毁，全局 y 不受影响

int main() {
    cout << "x from main " << x << "\n";
    cout << "y from main " << y << "\n";

    myFunction();

    cout << "Back from myFunction!\n\n";
    cout << "x from main " << x << "\n";
    cout << "y from main " << y << "\n";

    return 0;
}
```

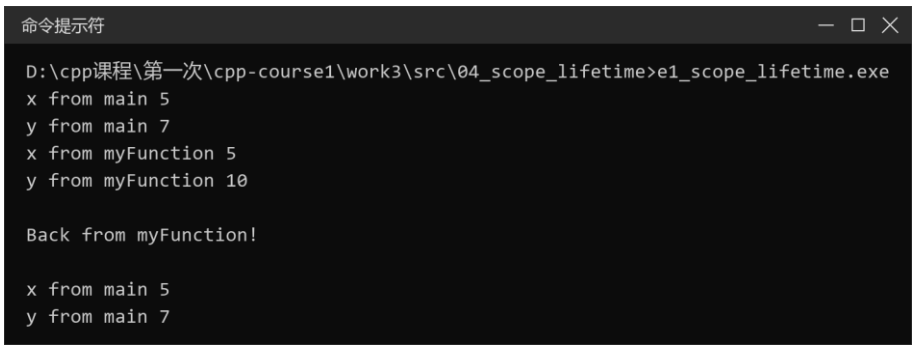


图 4-1 全局变量与局部变量的作用域运行结果

结果分析：运行结果与预期一致：main 中 x=5、y=7；myFunction 中 x=5、y=10；返回 main 后 y 仍是 7。myFunction 中的 int y=10 是局部变量，作用域只在函数体内，它遮蔽了同名的全局 y，但不会修改全局 y；局部 y 的生存期是函数调用期间，函数返回后即销毁。myFunction 中没有局部 x，所以访问的是全局 x。作用域、可见性和生存期是三个不同的概念：作用域是名字有效的代码范围，可见性还受同名遮蔽影响，生存期是对象实际存在的时间。

编程练习 5-7 Cat 类

1. 用静态数据成员统计 Cat 对象数目

实验内容：定义 Cat 类，静态数据成员 numOfCats 记录当前存在的 Cat 对象数目；构造函数加 1，析构函数减 1；静态成员函数 getNumOfCats() 读取 numOfCats。设计程序测试类名调用、对象调用和内层作用域析构。

代码：

【Cat.h】

```
#ifndef CAT_H
#define CAT_H

#include <string>

class Cat {
private:
    static int numOfCats;      // 静态数据成员：记录当前存在的 Cat 对象数目
    std::string name;          // 普通数据成员：每只猫自己的名字

public:
    Cat(const std::string &n = "cat");
    ~Cat();
    static int getNumOfCats(); // 静态成员函数：读取 numOfCats
    void show() const;
};

#endif  // CAT_H
```

【Cat.cpp】

```
#include <iostream>
#include "Cat.h"
using namespace std;

int Cat::numOfCats = 0;       // 静态数据成员在类外定义并初始化

Cat::Cat(const string &n) : name(n) {
    ++numOfCats;              // 每创建一个对象，总数加 1
    cout << "创建 Cat: " << name << "，当前数量 " << numOfCats << endl;
}

Cat::~Cat() {
    --numOfCats;              // 对象销毁时，总数减 1
    cout << "销毁 Cat: " << name << "，当前数量 " << numOfCats << endl;
}

int Cat::getNumOfCats() {
    return numOfCats;         // 静态成员函数直接访问静态数据成员
}

void Cat::show() const {
    // 普通成员函数既能访问普通成员 name，也能访问静态成员 numOfCats。
    cout << "我是 " << name << "，现在共有 " << numOfCats << " 只猫" << endl;
}
```

【main.cpp】

```
#include <iostream>
#include "Cat.h"
using namespace std;

int main() {
    // 没有对象时也能通过类名调用静态成员函数。
    cout << "程序开始时，猫的数量：" << Cat::getNumOfCats() << endl;

    Cat tom("Tom");
    Cat jerry("Jerry");
    cout << "创建两只猫后，猫的数量：" << Cat::getNumOfCats() << endl;
    tom.show();
    jerry.show();

    {
        Cat kitty("Kitty");
        cout << "内层作用域中，猫的数量：" << Cat::getNumOfCats() << endl;
        kitty.show();
    }   // kitty 离开作用域，析构函数被自动调用

    cout << "离开内层作用域后，猫的数量：" << Cat::getNumOfCats() << endl;
    cout << "通过对象调用静态成员函数：" << tom.getNumOfCats() << endl;

    return 0;
}
```

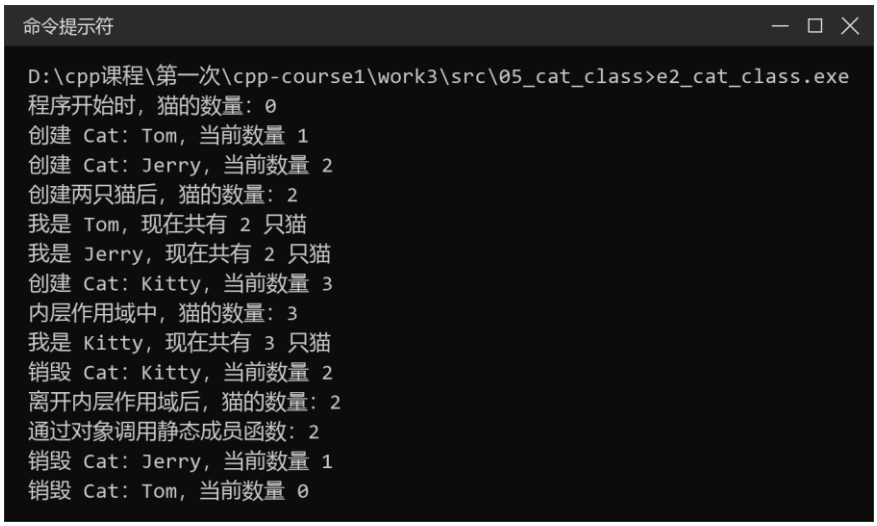


图 5-1 用静态数据成员统计 Cat 对象数目运行结果

结果分析：程序开始时猫的数量为 0；创建 Tom、Jerry 后 numOfCats 变为 2；内层作用域中创建 Kitty，数量变为 3，离开作用域时 Kitty 析构，数量回到 2；最后 main 结束，Jerry、Tom 依次析构，数量回到 0。numOfCats 是静态数据成员，所有 Cat 对象共享，所以任何一个对象构造或析构都会改变所有对象看到的计数。getNumOfCats() 是静态成员函数，没有对象时也能用 Cat::getNumOfCats() 调用，也可以用对象调用；show() 需要访问普通成员 name，因此必须是普通成员函数。此外，在 gdb 中对 Cat::Cat、Cat::~Cat、Cat::getNumOfCats、Cat::show 设置断点，可以看到构造顺序为 Tom→Jerry→Kitty，析构顺序为 Kitty→Jerry→Tom，与程序输出一致，说明局部对象的生存期由作用域决定。

编程练习 5-13 友元函数、友元成员函数与友元类

1. 三种友元对 X 私有成员的访问

实验内容：定义类 X、Y、Z：X 有私有成员 i；Y 的成员函数 g(X*) 是 X 的友元，对 i 加 1；Z 是 X 的友元类，其成员函数 f(X*) 对 i 加 5；全局函数 h(X*) 是 X 的友元，对 i 加 10。类声明与实现放在一个源文件组中，main() 放在另一个文件中。

代码：

【XYZ.h】

```
#ifndef XYZ_H
#define XYZ_H

class X;   // 前向声明：Y、Z 的成员函数只用到 X*，此处不必看到 X 的完整定义

// Y 的成员函数 g(X*) 将声明为 X 的友元成员函数
class Y {
public:
    void g(X *px);
};

// Z 将声明为 X 的友元类，Z 的所有成员函数都能访问 X 的私有成员
class Z {
public:
    void f(X *px);
};

class X {
private:
    int i;

public:
    X(int value = 0) : i(value) {}
    int getI() const { return i; }
    void show(const char *tag) const;

    friend void h(X *px);       // 全局函数 h 是 X 的友元函数
    friend void Y::g(X *px);    // Y 的成员函数 g 是 X 的友元成员函数
    friend class Z;             // Z 是 X 的友元类
};

void h(X *px);                  // 全局友元函数的声明

#endif  // XYZ_H
```

【XYZ.cpp】

```
#include <iostream>
#include "XYZ.h"
using namespace std;

// 友元成员函数：Y::g 可以直接访问 X 的私有成员 i
void Y::g(X *px) {
    px->i += 1;
}

// 友元类：Z 的所有成员函数都可以访问 X 的私有成员 i
void Z::f(X *px) {
    px->i += 5;
}

// 友元函数：h 不是任何类的成员，但可以访问 X 的私有成员 i
void h(X *px) {
    px->i += 10;
}

void X::show(const char *tag) const {
    cout << tag << ": i = " << i << endl;
}
```

【main.cpp】

```
#include <iostream>
#include "XYZ.h"
using namespace std;

int main() {
    X x(0);    // X 的私有成员 i 初值为 0
    Y y;
    Z z;

    x.show("初始值");

    y.g(&x);
    x.show("Y::g 加 1 后");

    z.f(&x);
    x.show("Z::f 加 5 后");

    h(&x);
    x.show("h 加 10 后");

    return 0;
}
```



图 6-1 三种友元对 X 私有成员的访问运行结果

结果分析：X 的私有成员 i 初值为 0；Y::g 加 1 后 i=1；Z::f 加 5 后 i=6；h 加 10 后 i=16，说明三种友元都能访问 X 的私有成员。friend void Y::g(X*) 要求 Y::g 在 X 之前声明，因此程序先前向声明 class X，再定义 Y、Z，最后定义 X；friend class Z 让 Z 的所有成员函数都成为 X 的友元；h 是全局函数，不是任何类的成员，所以没有 this 指针，需要把 X 对象用指针传进来。友元关系是单向的、不能传递，也不会被继承，使用时要注意它只是有限度地开放封装。多文件结构上，XYZ.h 放类声明，XYZ.cpp 放成员函数和友元函数的实现，main.cpp 只负责测试；编译时必须把 XYZ.cpp 和 main.cpp 一起交给 g++，否则链接阶段会报 undefined reference。

思考题 5-16 编译与链接

1. 编译、链接的输入输出与错误发生阶段

实验内容：课件末尾只列出“5-16、”而未展开题干。按教材第 5 章习题，5-16 是编译与链接的分析题，下面给出书面回答。

结果分析：（1）编译的输入是源文件（.cpp 及展开后的 .h），输出是目标文件（.o/.obj）；编译阶段完成词法、语法、语义检查，把源程序翻译成机器指令，生成可重定位目标文件。链接的输入是一个或多个目标文件以及库文件，输出是可执行文件（.exe）；链接阶段完成符号解析和地址重定位，把多个目标文件与库函数拼接成完整程序。

（2）① 定义 void f(int x, int y) 却以 f(1) 调用：编译阶段报错，实参个数少于形参个数。② 声明 void f(int x) 但未定义，且以 f(1) 调用：编译通过,链接阶段报错 undefined reference to 'f(int)'。③ 声明 void f(int x) 但未定义，也未调用：编译、链接都通过，因为没有生成对 f 的引用，链接器不需要找到定义。④ a.cpp 和 b.cpp 都定义 void f(int x)，分别编译后链接：各自编译通过，链接阶段报 multiple definition of 'f(int)'，违反单一定义规则（ODR）。

四、实验总结（本次实验的收获、体会、经验和教训）

本次实验围绕“数据的共享与保护”展开，完成了静态数据成员、静态数据成员与普通数据成员的区别、静态成员函数找错、变量作用域与生存期、Cat 类、友元与多文件结构共 6 个程序，并书面回答了编译与链接的思考题。所有程序均用 g++ -std=c++11 -Wall -Wextra -fexec-charset=GBK 编译通过并实际运行，运行结果保存在 work3/screenshots 中。

静态数据成员最大的特点是“属于类而不属于对象”。Ctype 的 s 和 Student 的 count 都只有一份存储，所有对象共享；普通成员 a、StudentNo 则每个对象各有一份。静态数据成员必须在类外定义并初始化，定义时前面不能加 static；如果只在类内声明而不在类外定义，链接时会报 undefined reference。这个规则一开始容易漏写，编译能通过、链接才报错，正好和 5-16 的编译/链接分析互相印证。

静态成员函数没有 this 指针，只能直接访问静态成员，这是我本次实验印象最深的一点。课件原程序在 static void Print() 中写 ++a，编译直接报 invalid use of member 'Ctype::a' in static member function；改成只访问 s 后，Ctype::Print() 可以在没有任何对象时通过类名调用。如果确实要访问某个对象的普通成员，可以把对象作为参数传入，或者改用普通成员函数。静态成员函数既可以用类名调用，也可以用对象调用，但本质上都不需要 this 指针。

作用域、可见性和生存期是三个容易混淆的概念。5-3 中 myFunction 的局部 y 遮蔽了全局 y，但函数返回后全局 y 仍是 7；这说明局部变量的作用域只在函数体内，生存期也只是函数调用期间，而同名遮蔽只影响可见性，不会改变全局变量的值。把这些概念和静态局部变量放在一起对比，理解会更清楚。

多文件结构是本次实验的另一个重点。每个题目都按“类声明 .h、类实现 .cpp、测试 main.cpp”拆分，头文件用 #ifndef/#define/#endif 防止重复包含；编译时要把同一目录下所有 .cpp 一起交给 g++，只编译 main.cpp 会在链接阶段报 undefined reference。5-13 的友元还涉及声明顺序：friend void Y::g(X*) 要求 Y::g 先声明，X 先前向声明，否则编译器看不到 Y::g 或 X 的完整信息。友元关系是单向的、不能传递、不会被继承，它只是有限度地开放封装，不能滥用。

本次实验的不足是：写静态成员时曾忘记在类外定义 s，导致链接错误；5-13 中 friend 声明的顺序也调试了几次。后续会继续做到：先画清楚类与类之间的关系，再写声明；编译时打开 -Wall -Wextra；遇到链接错误先区分“未定义”还是“重复定义”；每写完一个程序都用小例子验证静态成员的共享效果。